

Data Visualization assignment

February 13, 2025

0.1 Name : Raghul Khanna

0.2 1. Import the attached Indian_food.csv file in Jupyter notebook and perform following data cleaning operations through operations using Pandas and visualizations using Matplotlib, Seaborn and Plotly:

```
[17]: import pandas as pd
```

```
[40]: import os
      print(os.listdir())
```

```
['.ipynb_checkpoints', 'Assigment_Python_Basics.docx', 'Assignment -4',
'Assignment -4.zip', 'Assignment Pandas.pdf', 'Assignment submission',
'Assignment.docx', 'Billionaires_Stats.csv', 'Data Visualization
assignment.ipynb', 'indian_food.csv', 'marks.xlsm', 'Mini projects',
'netflix_title.xlsx', 'Numpy assignment.ipynb', 'Pandas Assignment (1)', 'Pandas
Assignment (1).zip', 'Pandas Assignment.ipynb', 'Student Performance Analysis
(1).docx', 'students.xlsm', 'test.csv']
```

```
[23]: df = pd.read_csv('indian_food.csv')
      df.info(), df.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 253 entries, 0 to 252
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Name                  253 non-null   object
1   Ingredients            253 non-null   object
2   Diet                  253 non-null   object
3   cook_time              253 non-null   int64
4   flavor_profile         253 non-null   object
5   course                253 non-null   object
6   state                 253 non-null   object
7   region                253 non-null   object
dtypes: int64(1), object(7)
memory usage: 15.9+ KB
```

[23]: (None,

	Name	Ingredients	\
0	Balu shahi	Maida flour, yogurt, oil, sugar	
1	Boondi	Gram flour, ghee, sugar	
2	Gajar ka halwa	Carrots, milk, sugar, ghee, cashews, raisins	
3	Ghevar	Flour, ghee, kewra, milk, clarified butter, su...	
4	Gulab jamun	Milk powder, plain flour, baking powder, ghee,...	

	Diet	cook_time	flavor_profile	course	state	region
0	vegetarian	25	sweet	dessert	West Bengal	East
1	vegetarian	30	sweet	dessert	Rajasthan	West
2	vegetarian	60	sweet	dessert	Punjab	North
3	vegetarian	30	sweet	dessert	Rajasthan	West
4	vegetarian	40	sweet	dessert	West Bengal	East

[]:

0.3 a) In the state column, replace the -1 with the mode of state column.

0.4 b) Whatever the state you got in previous question, fill the corresponding region in the region column.

0.5 c) Replace the -1 with mean for cook time variable.

```
[36]: # a)
mode_state = df[df['state'] != '-1']['state'].mode()[0]
df['state'] = df['state'].replace('-1', mode_state)

# b)
mode_state_region = df[df['state'] == mode_state]['region'].mode()[0]
df.loc[df['state'] == mode_state, 'region'] = mode_state_region

# c)
mean_cook_time = df[df['cook_time'] != -1]['cook_time'].mean()
df['cook_time'] = df['cook_time'].replace(-1, mean_cook_time)

df[['state', 'region', 'cook_time']].head()
```

```
[36]:      state region  cook_time
0  West Bengal  East      25.0
1   Rajasthan  West      30.0
2    Punjab  North      60.0
3   Rajasthan  West      30.0
4  West Bengal  East      40.0
```

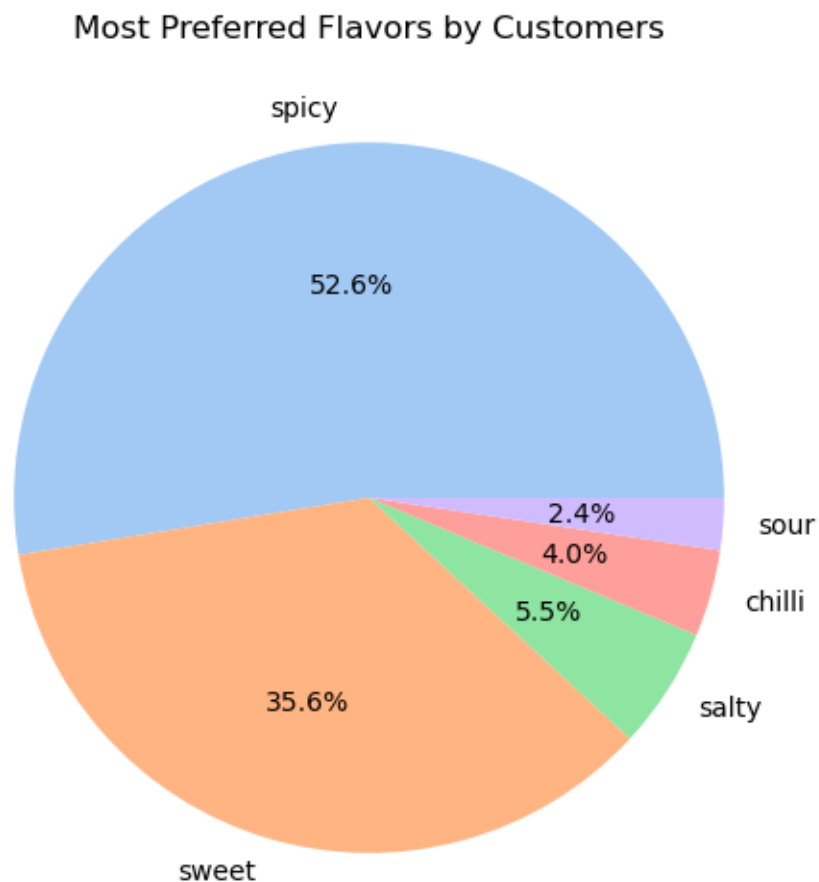
[]:

0.6 After performing these data cleaning steps, perform the below visualizations. You can use any of the 3 libraries (Matplotlib, seaborn, Plotly):

0.7 d) Most preferred flavours by number of customers using pie chart.

```
[37]: import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px

flavor_counts = df['flavor_profile'].value_counts()
plt.figure(figsize=(8, 6))
plt.pie(flavor_counts, labels=flavor_counts.index, autopct='%1.1f%%',
        colors=sns.color_palette("pastel"))
plt.title("Most Preferred Flavors by Customers")
plt.show()
```



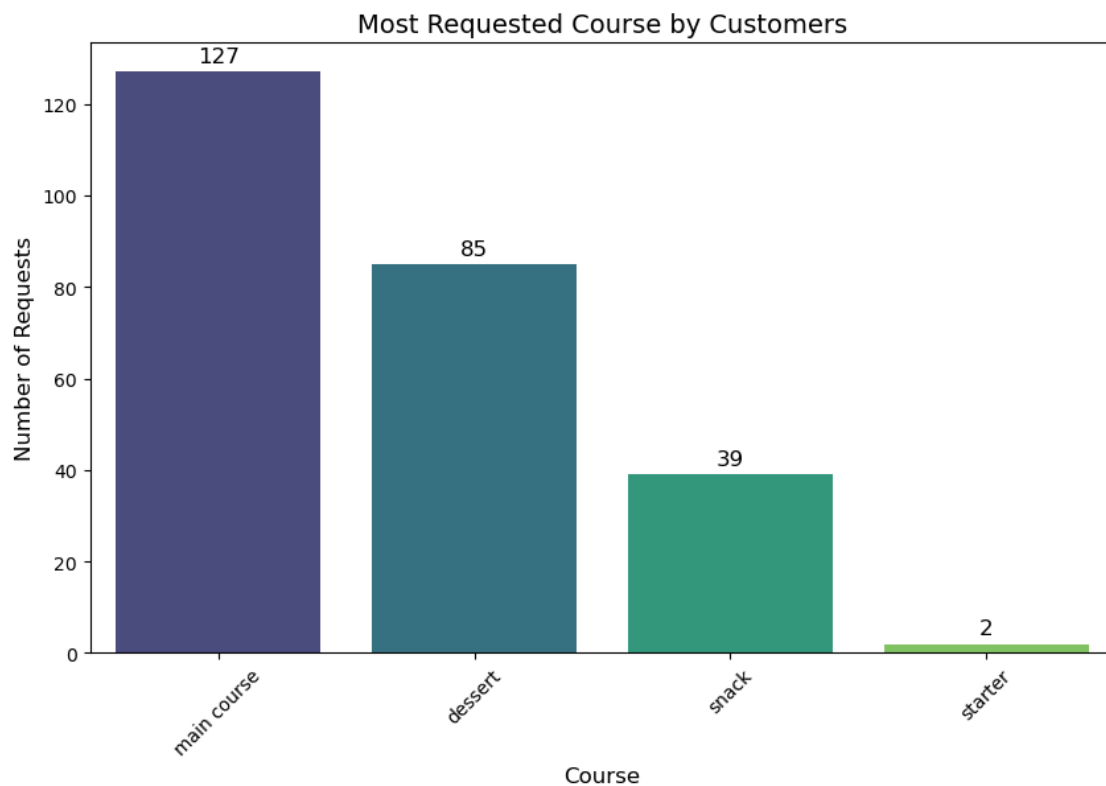
```
[ ]:
```

0.8 e) Most requested course by customers using Bar chart. Print the labels also on the top of each bar. Then sort the bar chart in descending order.

```
[74]: plt.figure(figsize=(10, 6))
ax = sns.barplot(x=course_counts.index, y=course_counts.values,
                hue=course_counts.index, palette="viridis", legend=False)

for i, v in enumerate(course_counts.values):
    ax.text(i, v + 2, str(v), ha='center', fontsize=12)

plt.title("Most Requested Course by Customers", fontsize=14)
plt.xlabel("Course", fontsize=12)
plt.ylabel("Number of Requests", fontsize=12)
plt.xticks(rotation=45)
plt.show()
```



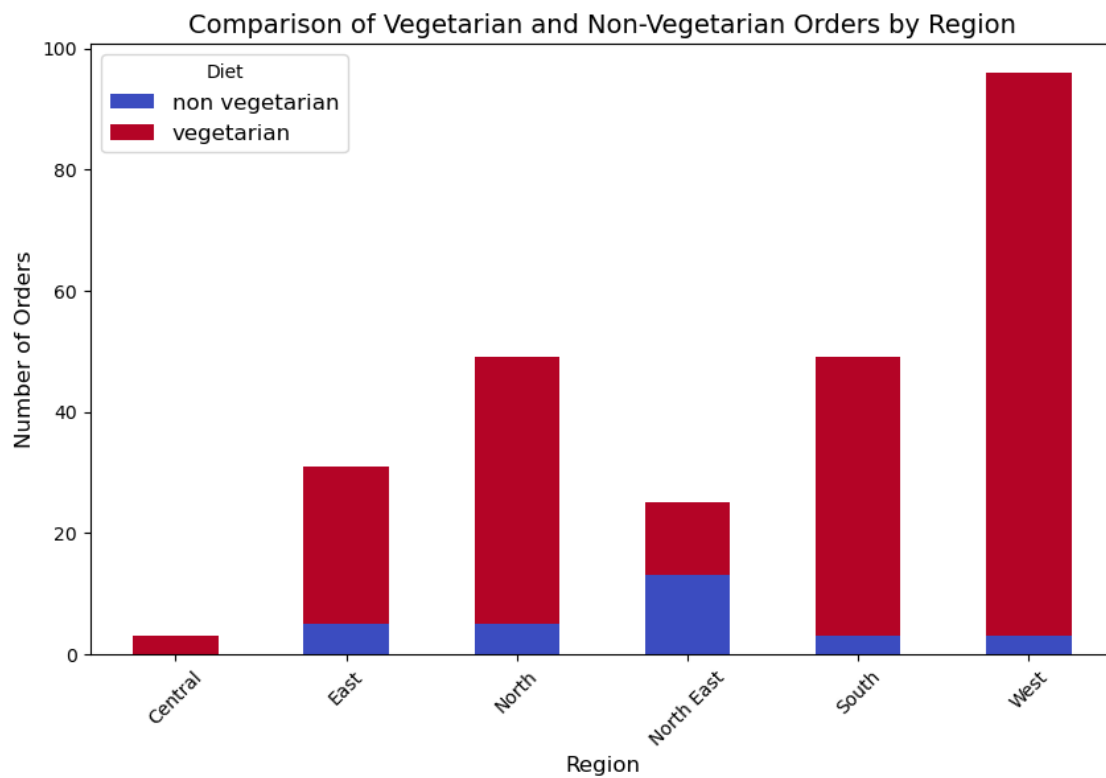
```
[ ]:
```

0.9 f) Comparison between Vegetarians and Non-Veg orders in each region.
Hint: You can use stacked/grouped bar chart for comparison.

```
[39]: diet_region_counts = df.groupby(['region', 'Diet']).size().unstack()

diet_region_counts.plot(kind='bar', stacked=True, figsize=(10, 6),
    colormap='coolwarm')

plt.title("Comparison of Vegetarian and Non-Vegetarian Orders by Region",
    fontsize=14)
plt.xlabel("Region", fontsize=12)
plt.ylabel("Number of Orders", fontsize=12)
plt.legend(title="Diet", fontsize=12)
plt.xticks(rotation=45)
plt.show()
```

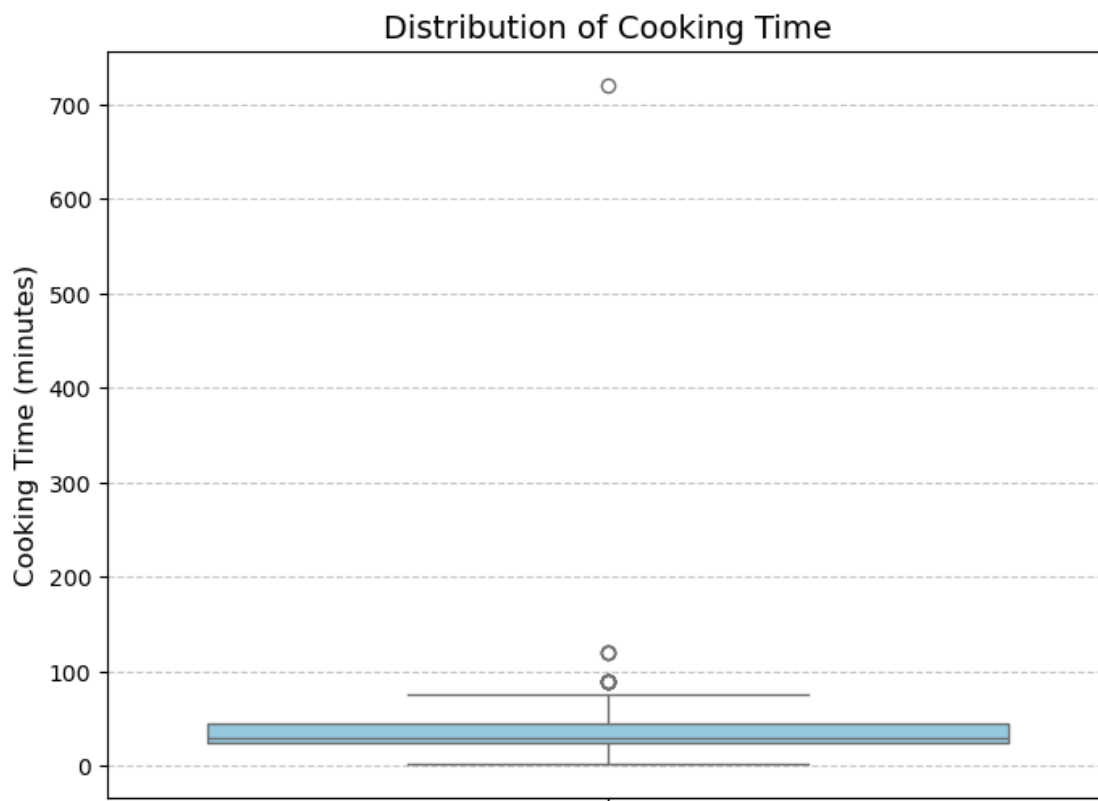


```
[ ]:
```

0.10 g) Create the box plot for Order cooking time to see the distribution. Also check if there are any outliers in this column.

```
[14]: plt.figure(figsize=(8, 6))
sns.boxplot(y=df['cook_time'], color='skyblue')

plt.title("Distribution of Cooking Time", fontsize=14)
plt.ylabel("Cooking Time (minutes)", fontsize=12)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```



```
[ ]:
```

```
[ ]:
```

0.11 2. Import the attached Billionaires_Stats.csv file in Jupyter notebook and perform following operations using Pandas and Data visualization:

```
[44]: import os
print(os.listdir())
```

```
['.ipynb_checkpoints', 'Assignment_Python_Basics.docx', 'Assignment -4',
```

```
'Assignment -4.zip', 'Assignment Pandas.pdf', 'Assignment submission',
'Assignment.docx', 'Billionaires_Stats.csv', 'billionaires_Stats.xlsx', 'Data
Visualization assignment.ipynb', 'indian_food.csv', 'marks.xlsm', 'Mini
projects', 'netflix_title.xlsx', 'Numpy assignment.ipynb', 'Pandas Assignment
(1)', 'Pandas Assignment (1).zip', 'Pandas Assignment.ipynb', 'Student
Performance Analysis (1).docx', 'students.xlsm', 'test.csv']
```

```
[47]: billionaires_df = pd.read_excel('billionaires_Stats.xlsx')
```

```
billionaires_df.info(), billionaires_df.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2640 entries, 0 to 2639
Data columns (total 10 columns):
#   Column          Non-Null Count  Dtype
---  -
0   finalWorth      2640 non-null   int64
1   category        2640 non-null   object
2   personName      2640 non-null   object
3   age             2575 non-null   float64
4   country         2602 non-null   object
5   city            2568 non-null   object
6   gender          2640 non-null   object
7   birthYear       2564 non-null   float64
8   birthMonth      2564 non-null   float64
9   birthDay        2564 non-null   float64
dtypes: float64(4), int64(1), object(5)
memory usage: 206.4+ KB
```

```
[47]: (None,
      finalWorth      category      personName  age  \
0      211000      Fashion & Retail  Bernard Arnault & family  74.0
1      180000      Automotive      Elon Musk  51.0
2      114000      Technology      Jeff Bezos  59.0
3      107000      Technology      Larry Ellison  78.0
4      106000  Finance & Investments  Warren Buffett  92.0

      country  city gender  birthYear  birthMonth  birthDay
0      France  Paris    M    1949.0         3.0         5.0
1  United States  Austin    M    1971.0         6.0        28.0
2  United States  Medina    M    1964.0         1.0        12.0
3  United States  Lanai    M    1944.0         8.0        17.0
4  United States  Omaha    M    1930.0         8.0       30.0 )
```

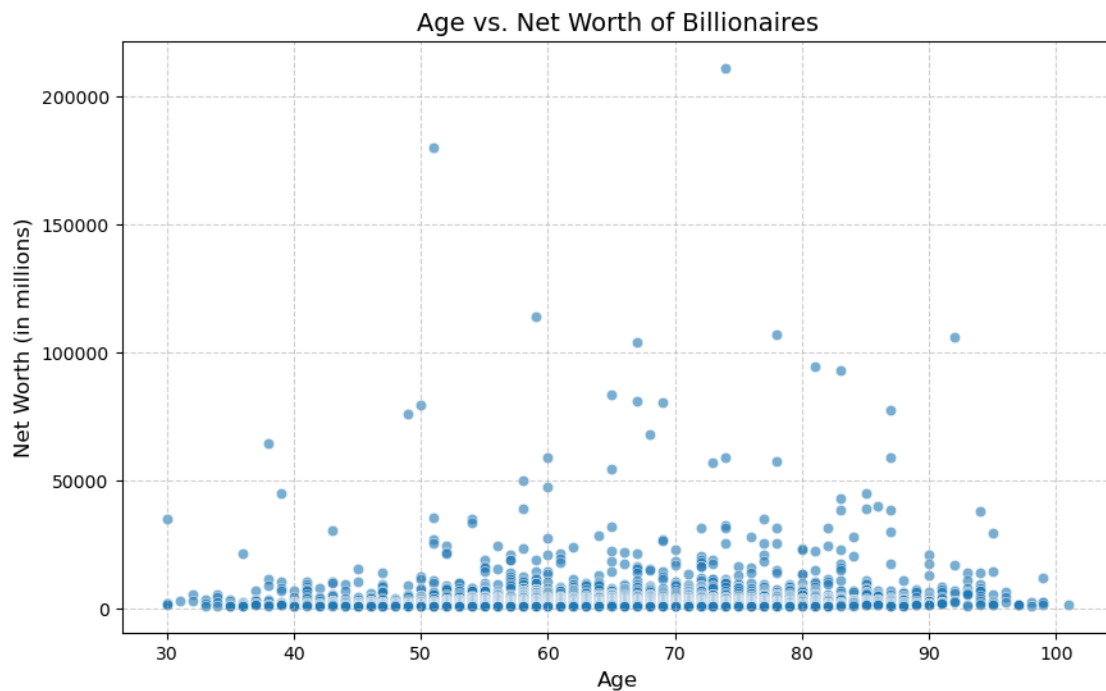
```
[ ]:
```

0.12 a) Create a dataframe which will have the billionaires whose age is greater than or equal to 30. Then create a scatter plot to show the relationship between age and net worth.

```
[49]: billionaires_30_plus = billionaires_df[billionaires_df['age'] >= 30]

plt.figure(figsize=(10, 6))
sns.scatterplot(x=billionaires_30_plus['age'], y=billionaires_30_plus['finalWorth'], alpha=0.6)

plt.title("Age vs. Net Worth of Billionaires", fontsize=14)
plt.xlabel("Age", fontsize=12)
plt.ylabel("Net Worth (in millions)", fontsize=12)
plt.grid(True, linestyle="--", alpha=0.6)
plt.show()
```



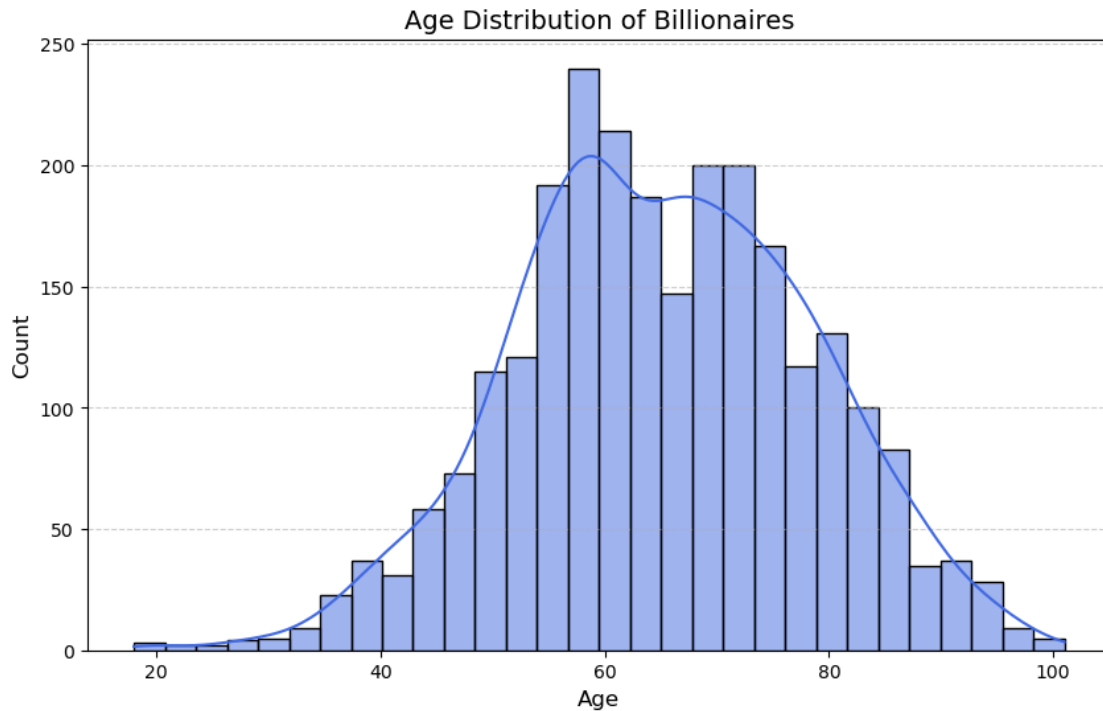
```
[ ]:
```

0.13 b) Create a histogram to see the distribution of ages.

```
[50]: plt.figure(figsize=(10, 6))
sns.histplot(billionaires_df['age'].dropna(), bins=30, kde=True, color="royalblue")
```



```
plt.title("Age Distribution of Billionaires", fontsize=14)
plt.xlabel("Age", fontsize=12)
plt.ylabel("Count", fontsize=12)
plt.grid(axis='y', linestyle='--', alpha=0.6)
plt.show()
```



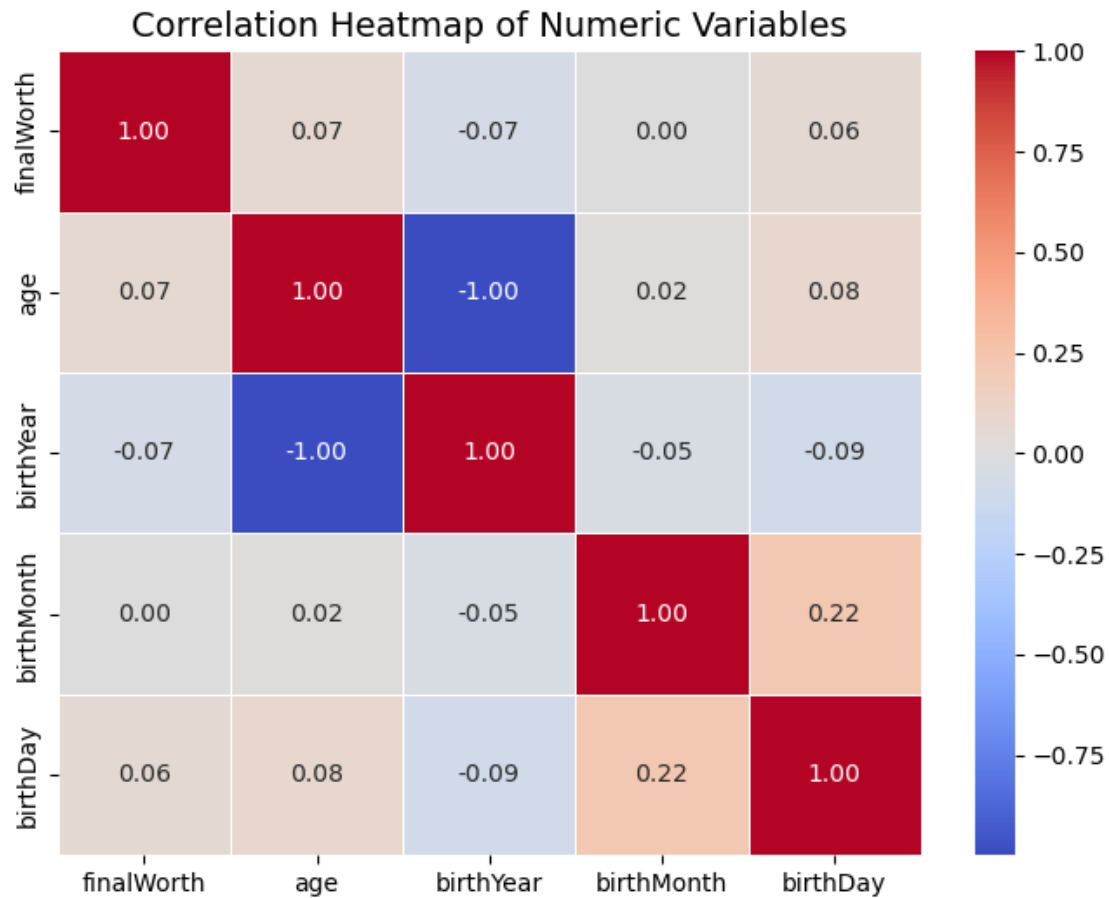
[]:

0.14 c) Create a heatmap and see the correlation between all the numeric variables. Print the correlation value also in the heatmap.

```
[65]: plt.figure(figsize=(8, 6))
correlation_matrix = billionaires_df.corr(numeric_only=True)

sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm", fmt=".2f",
            line widths=0.5)

plt.title("Correlation Heatmap of Numeric Variables", fontsize=14)
plt.show()
```



[]:

0.15 d) Save the figure in your pwd generated through previous question.

[54]: `pwd`

[54]: `'D:\\Raghul Khanna\\Python\\1. ASSIGNMENTS'`

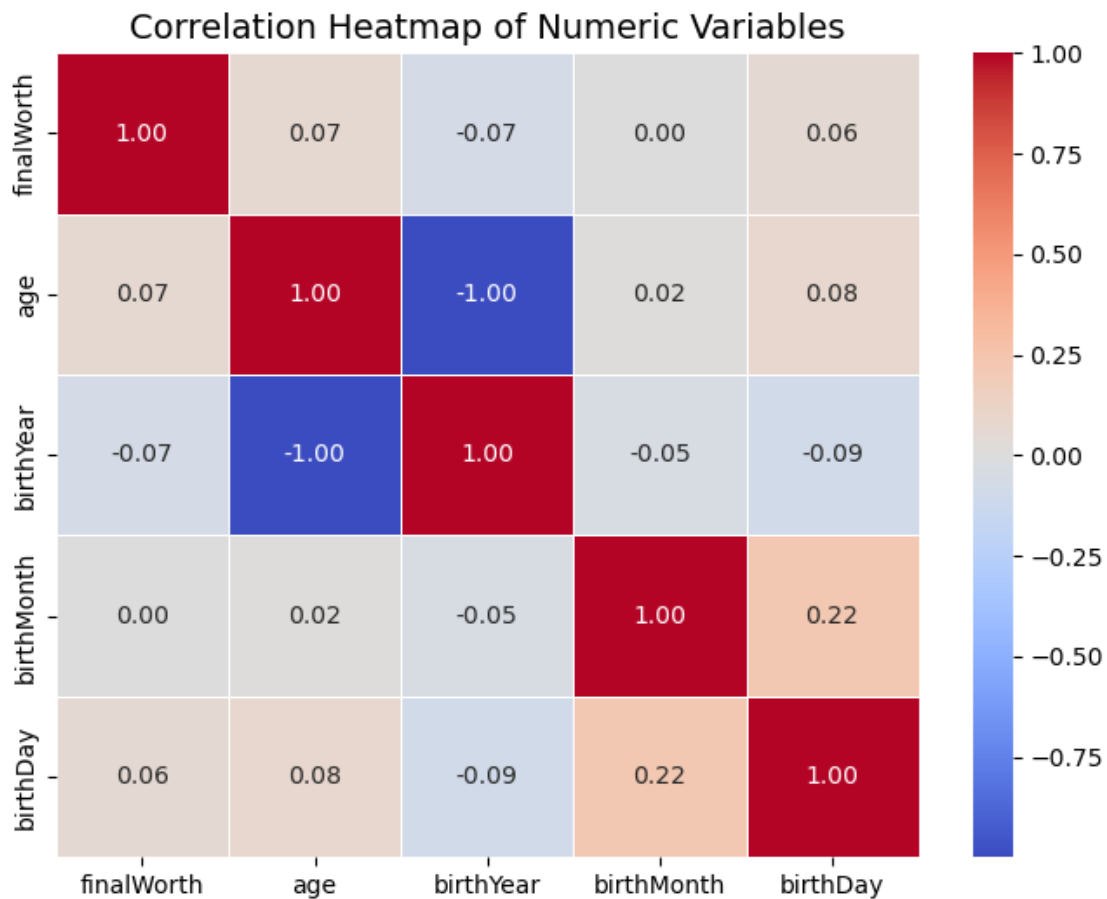
```
[68]: current_directory = os.getcwd()

file_name = "correlation_matrix.png"
file_path = os.path.join(current_directory, file_name)
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm", fmt=".2f",
            linewidths=0.5)
plt.title("Correlation Heatmap of Numeric Variables", fontsize=14)
plt.savefig(file_path)
```

```
print(f"Figure saved to: {file_path}")

plt.show()
```

Figure saved to: D:\Raghul Khanna\Python\1. ASSIGNMENTS\correlation_matrix.png



[]:

[]:

0.16 3. Name any 5 types of charts you are familiar with (using any of 3 libraries).

0.17 Write the sample code also to create those charts using random data.

```
[73]: import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import pandas as pd
```

```

np.random.seed(50)
x = np.linspace(0, 10, 50)
y = np.sin(x) + np.random.normal(0, 0.2, 50)
z = np.cos(x) + np.random.normal(0, 0.2, 50)
categories = ['A', 'B', 'C', 'D', 'E']
values = np.random.randint(20, 100, 5)
data = {'Category': categories, 'Value': values}
df = pd.DataFrame(data)

# 1. Line Chart (Matplotlib)
plt.figure(figsize=(5, 5))
plt.plot(x, y, label='Series 1', marker='o')
plt.plot(x, z, label='Series 2', marker='x')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.title('Line Chart')
plt.legend()
plt.grid(True)
plt.show()

# 2. Bar Chart (Matplotlib)
plt.figure(figsize=(5, 5))
plt.bar(df['Category'], df['Value'], color='skyblue')
plt.xlabel('Categories')
plt.ylabel('Values')
plt.title('Bar Chart')
plt.show()

# 3. Scatter Plot (Matplotlib)
plt.figure(figsize=(5, 5))
plt.scatter(x, y, label='Data Points', color='green', marker='.')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.title('Scatter Plot')
plt.legend()
plt.grid(True)
plt.show()

# 4. Box Plot (Seaborn)
plt.figure(figsize=(5, 5))
sns.boxplot(x='Category', y='Value', hue='Category', data=df, palette='Set3',
            legend=False) # Added hue and legend=False
plt.xlabel('Categories')
plt.ylabel('Values')
plt.title('Box Plot (Seaborn)')

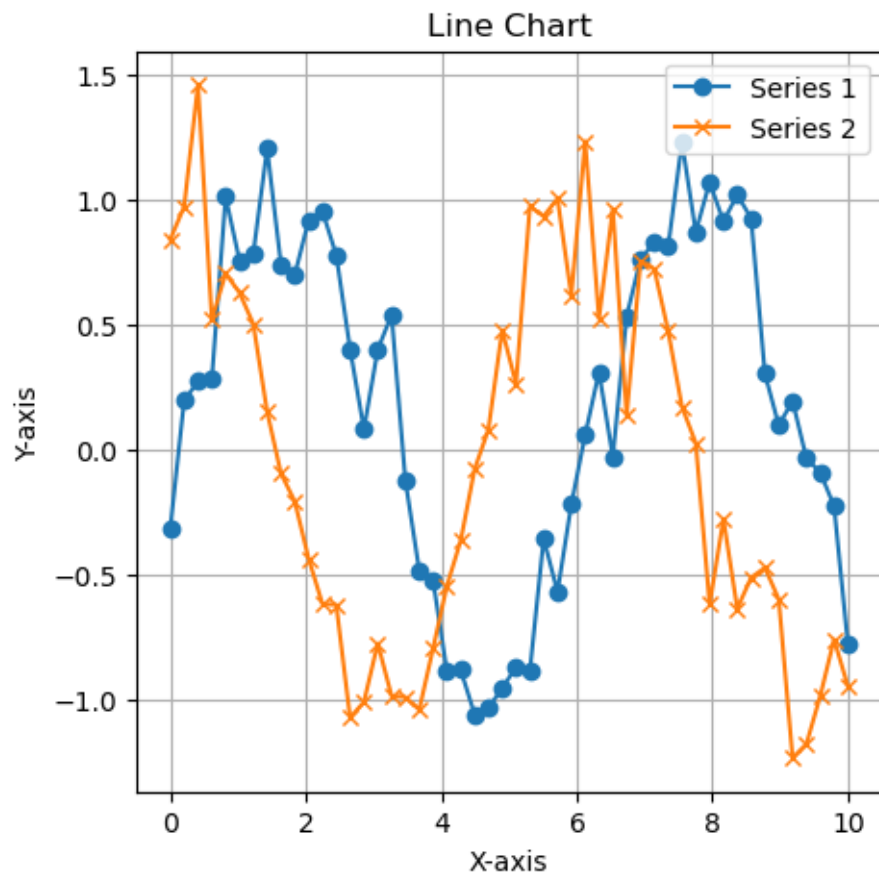
```

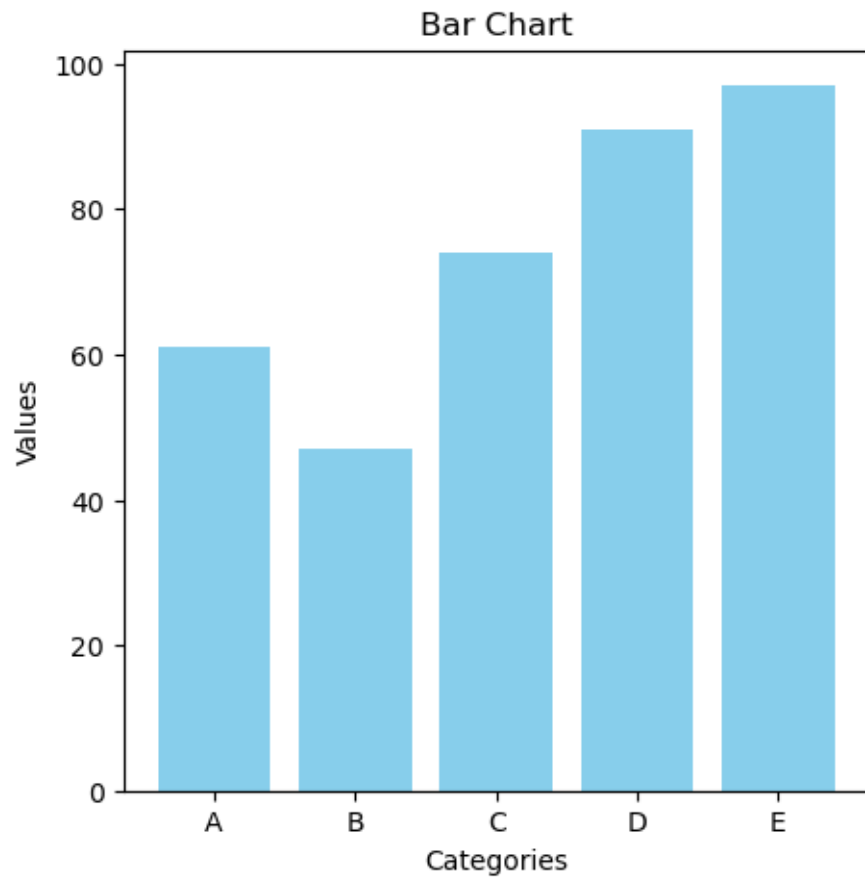
```
plt.show()
```

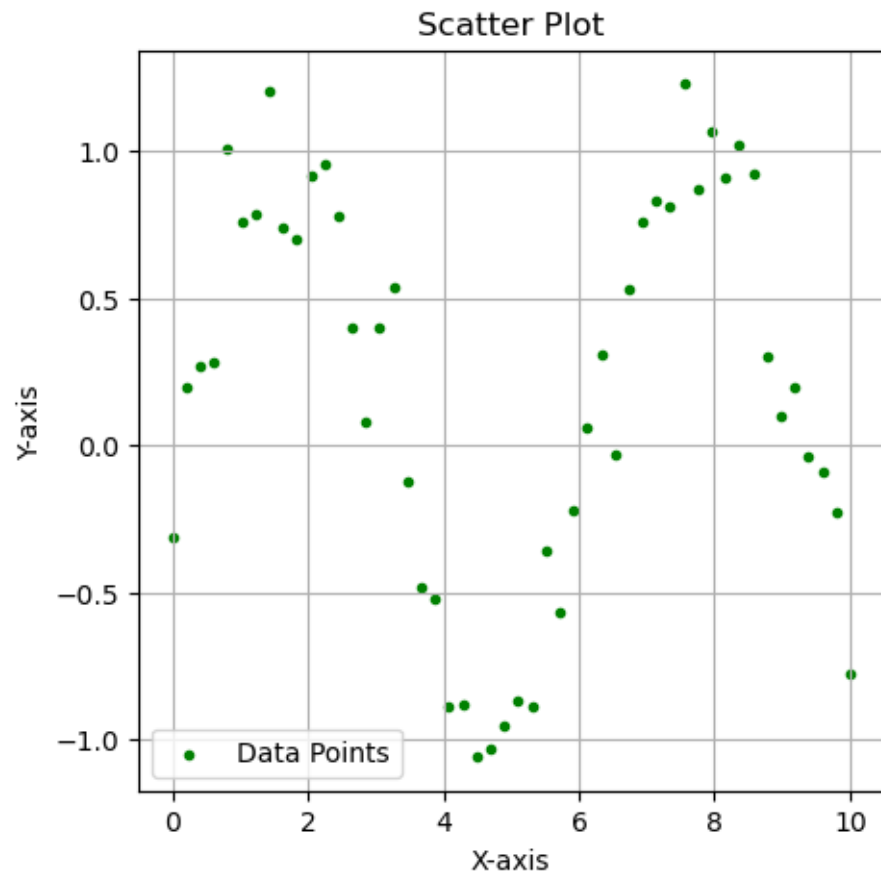
```
# 5. Box Plot (Plotly)
```

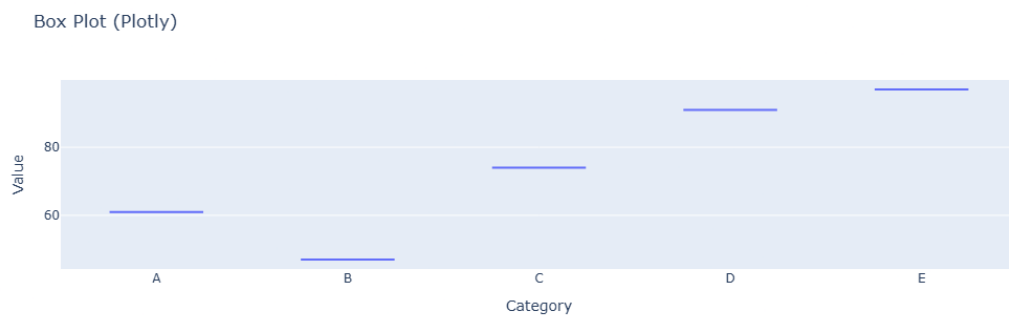
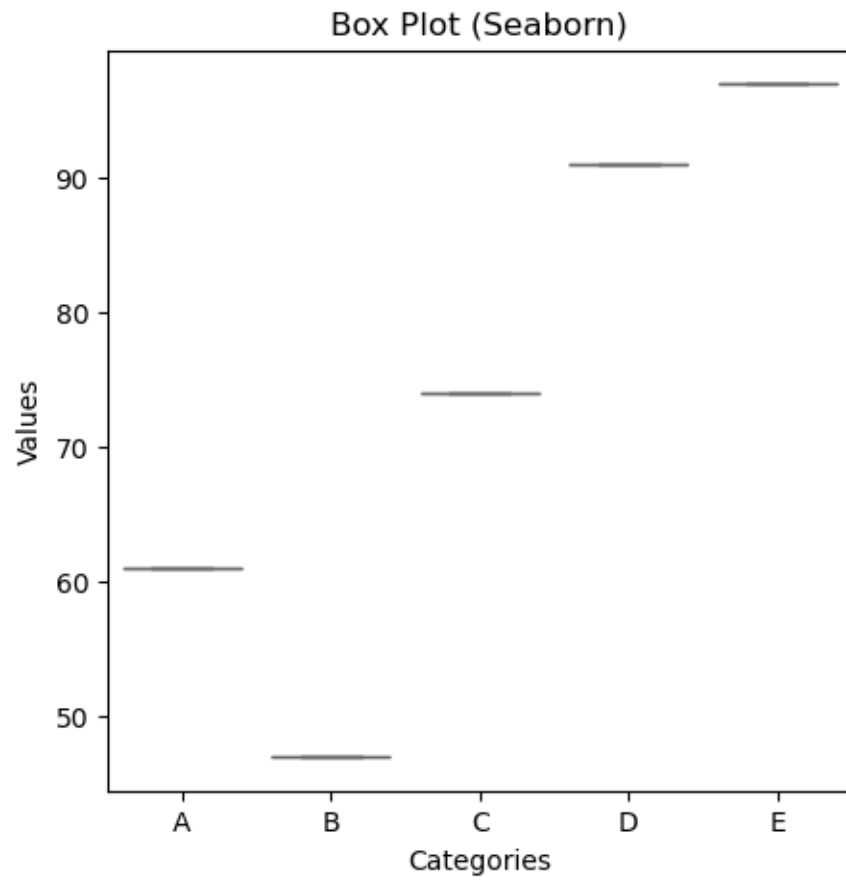
```
fig_box = px.box(df, x='Category', y='Value', title='Box Plot (Plotly)')
```

```
fig_box.show()
```









[]: